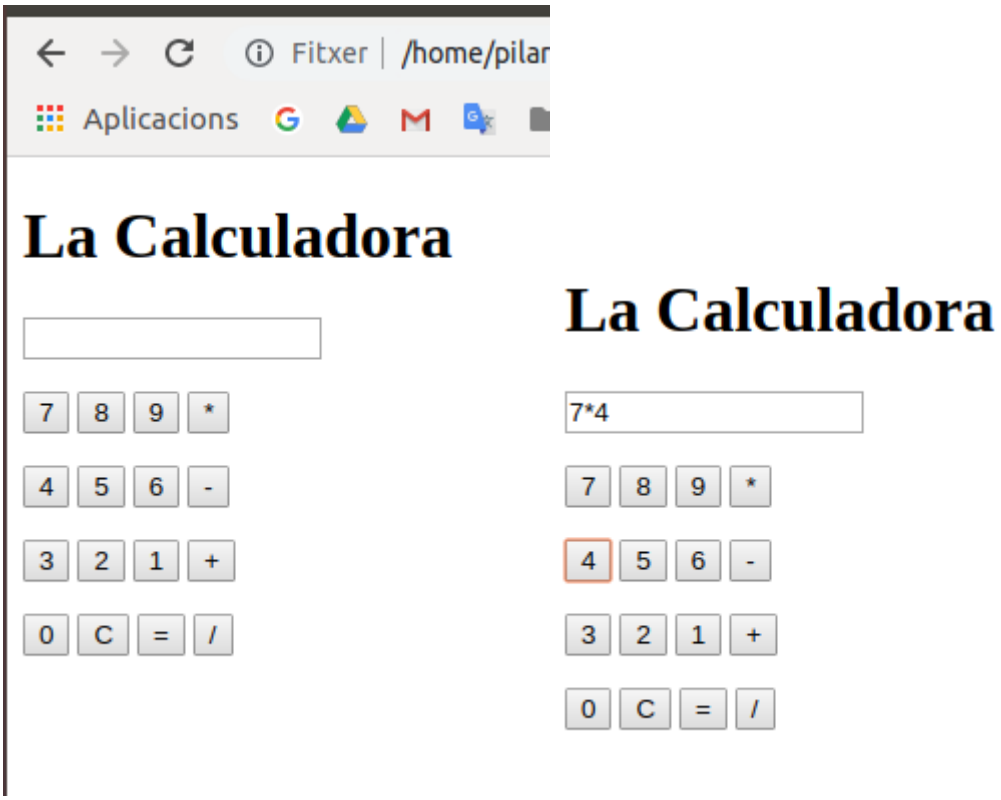
Exercicis amb el DOM-BOM i TIMERS,

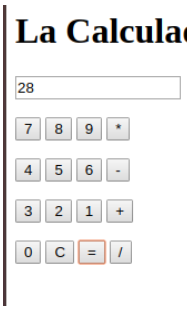
1.- Cal que feu una pàgina web que simularà una calculadora:



Podeu utilitzar la funció eval(), que avalua un string matemàticament.

eval("4+3") retorna 7

La idea és fer un string que concatena tots els caràcters dels botons que són premuts i quan es prem l'igual s'avalua. També es poden utilitzar les tecles numèriques i d'operació per escriure el valor al display de la calculadora.



2.- Sota la calculadora ha d'aparèixer un cronòmetre que indiqui quantes hores, minuts i segons hem treballat. S'ha d'actualitzar cada segon.

3.- Simuleu un teclat de piano (una única octava) amb el nom de la nota a cada tecla (en anglès). Cal que la pressió sobre alguna de les tecles emeti la nota musical corresponent. La pressió es pot fer tocant la tecla corresponent a la nota o bé amb el punter del ratolí en fer *click* sobre. Es valorarà la creativitat…

Pista: Utilitzeu la Web Audio API, amb es 3 funcions següents.

```
const FREQUENCIES = {
     'C4': 261.63,
     'C#4': 277.18,
     'D4': 293.66,
     'D#4': 311.13,
     'E4': 329.63,
     'F4': 349.23,
     'F#4': 369.99,
     'G4': 392.00,
     'G#4': 415.30,
     'A4': 440.00,
     'A#4': 466.16,
     'B4': 493.88,
     'C5': 523.25
};


let audioCtx = null;
const activeNotes = new Map();

// Initialize AudioContext
function initAudio() {
    if (!audioCtx) {
        audioCtx = new (window.AudioContext ||
window.webkitAudioContext)();
    }
    if (audioCtx.state === 'suspended') {
        audioCtx.resume();
    }
}

function playNote(note) {
    initAudio();
    if (activeNotes.has(note)) return;

    const freq = FREQUENCIES[note];
```

```
            if (!freq) return;

            const osc = audioCtx.createOscillator();
            const gain = audioCtx.createGain();

            osc.type = 'triangle';
            osc.frequency.setValueAtTime(freq, audioCtx.currentTime);

            // Smooth attack
            gain.gain.setValueAtTime(0.001, audioCtx.currentTime);
            gain.gain.linearRampToValueAtTime(0.5, audioCtx.currentTime +
0.02);

            osc.connect(gain);
            gain.connect(audioCtx.destination);
            osc.start();

            activeNotes.set(note, { osc, gain });

            // UI highlight
            const keyEl = document.querySelector(`[data-note="${note}"]`);
            if (keyEl) keyEl.classList.add('active');
        }

        function stopNote(note) {
            if (!activeNotes.has(note)) return;

            const { osc, gain } = activeNotes.get(note);
            const now = audioCtx.currentTime;

            // Smooth release
            gain.gain.cancelScheduledValues(now);
            gain.gain.setValueAtTime(gain.gain.value, now);
            gain.gain.exponentialRampToValueAtTime(0.0001, now + 0.25);

            setTimeout(() => {
                try {
                    osc.stop();
                    osc.disconnect();
                } catch (e) {}
            }, 250);

            activeNotes.delete(note);
```

```
        // UI un-highlight
        const keyEl = document.querySelector(`[data-note="${note}"]`);
        if (keyEl) keyEl.classList.remove('active');
    }
```